

Les Tokens

1. Introduction : Qu'est-ce qu'un Token ?

Aujourd'hui, la majorité des applications web modernes utilisent un système appelé **Token d'authentification** pour gérer les connexions des utilisateurs.

Un token peut être comparé à un **bracelet de festival VIP**.

Exemple simple

Imaginez :

- Le **serveur** est un vigile.
- L'utilisateur est un festivalier.
- L'application Vue.js représente la personne qui montre le bracelet.

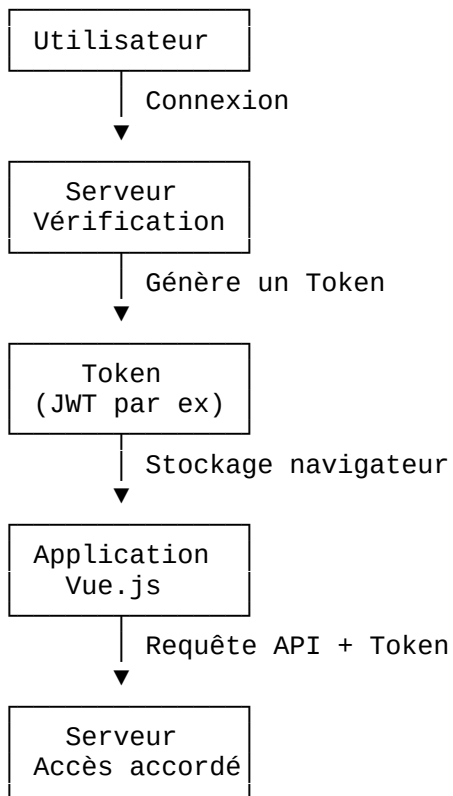
Sans token, l'utilisateur devrait :

- redonner son identifiant,
- son mot de passe,
- et prouver son identité à chaque action.

Avec un token :

1. L'utilisateur se connecte une seule fois.
 2. Le serveur vérifie les informations.
 3. Le serveur donne un "bracelet numérique" : le token.
 4. Ensuite, l'application montre simplement ce token pour accéder aux ressources.
-

Schéma de fonctionnement



2. Pourquoi utiliser un Token ?

Les tokens sont devenus indispensables dans les applications modernes.

Les avantages

Rapidité

Le serveur n'a pas besoin de vérifier les identifiants complets à chaque requête.

Il lit simplement le token envoyé.

Résultat :

- moins de calculs,
 - serveur plus rapide,
 - meilleure fluidité.
-

Fonctionnement sans rechargement

Les frameworks modernes comme :

- Vue.js,
- React,
- Angular

fonctionnent comme de véritables logiciels dans le navigateur.

Le token permet :

- de rester connecté,
 - sans recharger la page,
 - avec une expérience utilisateur fluide.
-

Compatible partout

Un même token peut être utilisé :

- sur un site web,
- une application mobile,
- plusieurs serveurs,
- une API externe.

Cela facilite énormément le développement moderne.

3. Les inconvénients et risques

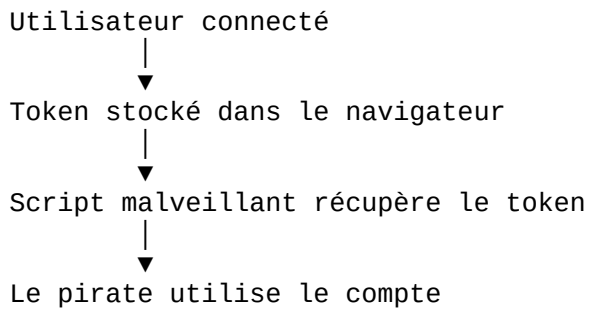
Même si les tokens sont très pratiques, ils présentent aussi certains risques.

Vol de Token (attaque XSS)

Si un pirate récupère le token stocké dans le navigateur, il peut se faire passer pour l'utilisateur.

C'est ce qu'on appelle souvent une faille **XSS (Cross Site Scripting)**.

Exemple



Difficulté de révocation

Un token reste souvent valide pendant une certaine durée.

Par exemple :

- 1 heure,
- 24 heures,
- ou plusieurs jours.

Si un utilisateur doit être déconnecté immédiatement :

- il est parfois compliqué d'invalider le token partout instantanément.
-

4. Pourquoi les Tokens sont devenus la norme ?

Les anciens sites web rechargeaient les pages en permanence.

Aujourd'hui, les applications web modernes fonctionnent différemment :

Exemples connus

- Netflix
- Spotify
- Gmail

Ces plateformes utilisent des systèmes d'authentification modernes basés sur des tokens.

Sans token :

- impossible d'avoir une expérience fluide,
 - impossible de maintenir une connexion rapide entre le client et le serveur.
-

5. Quand utiliser un système de Token ?

Le système de token est utilisé dans presque tous les projets modernes nécessitant une connexion utilisateur.

Exemples de projets

- Boutique en ligne
 - Réseau social
 - Tableau de bord administrateur
 - Application mobile
 - API REST
 - Jeu multijoueur
 - Plateforme SaaS
-

6. Fonctionnement technique

Étape 1 — Connexion

L'utilisateur envoie :

- son email,
- son mot de passe.

Le serveur vérifie les informations.

Étape 2 — Création du Token

Le serveur génère un token sécurisé.

Exemple de JWT : eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

Étape 3 — Stockage dans le navigateur

Le token est enregistré localement.

Exemple JavaScript

```
localStorage.setItem('token', 'votre_cle_secrete');
```

Étape 4 — Envoi automatique du Token

À chaque requête API, le token est envoyé dans les headers HTTP.

Exemple avec Axios

```
config.headers.Authorization = `Bearer ${token}`;
```

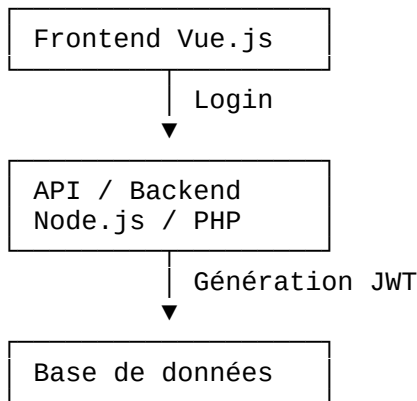
Étape 5 — Protection des pages

Avant d'ouvrir une page protégée, l'application vérifie la présence du token.

Exemple Vue Router

```
if (!token) {  
  next('/login');  
}
```

7. Architecture simplifiée d'un système Token



Puis : Frontend → Envoie le Token → API → Vérification → Réponse

8. Les bonnes pratiques de sécurité

Pour sécuriser un système de token :

À faire

- ✓ Utiliser HTTPS
 - ✓ Donner une durée de vie limitée au token
 - ✓ Vérifier les permissions côté serveur
 - ✓ Utiliser des refresh tokens
 - ✓ Nettoyer les données utilisateur
-

À éviter

- ✗ Stocker des données sensibles dans le token
 - ✗ Garder un token valide trop longtemps
 - ✗ Faire confiance uniquement au frontend
 - ✗ Ignorer les protections XSS
-

9. JWT : le format le plus utilisé

Le type de token le plus populaire aujourd'hui est le **JWT (JSON Web Token)**.

Un JWT contient :

- un en-tête,
- des informations utilisateur,
- une signature de sécurité.

Structure d'un JWT

HEADER . PAYLOAD . SIGNATURE

10. Conclusion

Les tokens sont devenus un élément essentiel du développement web moderne.

Ils permettent :

- une connexion rapide,
- une meilleure expérience utilisateur,
- une compatibilité avec les applications modernes,
- une architecture API performante.

Cependant, ils nécessitent :

- une bonne sécurisation,
- une gestion rigoureuse,
- et une attention particulière contre les failles de sécurité.

Dans des frameworks modernes comme Vue.js, les tokens représentent aujourd'hui la méthode standard pour gérer l'authentification utilisateur.

Sources utiles

Axios

[Documentation Axios Interceptors](#)

JWT

[Introduction officielle JWT](#)